Guide Frontend React

Contents

1 Objectifs **2**

2 Organisation du dépôt Git **2**
- 2.1 Structure cible . . . 2
- 2.2 Déplacer le backend (opération à faire une seule fois) . . . 2
- 2.3 Le fichier `.gitignore` à la racine . . . 2
- 2.4 Stratégie de branches . . . 3

3 Créer le projet React **4**
- 3.1 Création avec Vite . . . 4
- 3.2 Structure générée par Vite . . . 4
- 3.3 Commandes npm utiles . . . 4
- 3.4 Configurer le proxy de développement (indispensable) . . . 4
- 3.5 CORS : le point critique . . . 5

4 Architecture du frontend **6**
- 4.1 Arborescence recommandée . . . 6
- 4.2 Rôle de chaque dossier . . . 6
- 4.3 Ajouter le routage . . . 7
- 4.4 Conventions de nommage . . . 8

5 Communiquer avec le backend **8**
- 5.1 Principe : le frontend ne connaît que HTTP et JSON . . . 8
- 5.2 Contrat de l'API de gestion de stock . . . 8
- 5.3 Tester avec Postman avant de coder React . . . 9
- 5.4 La couche API du frontend . . . 9
- 5.5 Un hook pour charger les données . . . 10
- 5.6 Une page complète : liste + création + suppression . . . 11
- 5.7 Gérer les dates . . . 13

1 Objectifs

À la fin de ce guide, vous saurez :

- créer un projet React moderne avec **Vite** ;
- organiser le code frontend en couches claires (pages, composants, services API, hooks) ;
- appeler les routes REST du backend Java (GET, POST, PUT, DELETE) ;
- gérer les erreurs, le chargement et les dates correctement ;
- faire cohabiter `frontend/` et `backend/` dans un même dépôt Git et travailler sur chacun sans se mélanger.

Important : le backend doit déjà fonctionner seul. Si ce n'est pas le cas, corrigez d'abord le backend : le frontend ne peut pas remplacer un backend qui ne répond pas.

2 Organisation du dépôt Git

2.1 Structure cible

Le dépôt est un *monorepo* : un seul dépôt Git, deux applications indépendantes.

```
nom_projet/
|-- backend/                <-- application Java (Maven)
|   |-- pom.xml
|   |-- src/main/java/...
|   |-- src/main/resources/database.sql
|   `-- target/             (ignore par Git)
|-- frontend/               <-- application React (Vite / npm)
|   |-- package.json
|   |-- package-lock.json
|   |-- vite.config.js
|   |-- index.html
|   |-- .env.example
|   `-- src/
|-- .gitignore
`-- README.md
```

Remarque : normalement dans vos dépôts actuels, le backend est encore à la racine (`pom.xml`, `src/`). La structure ci-dessus est la cible à adopter. Le déplacement se fait une seule fois, puis tout le monde travaille dans la nouvelle structure.

2.2 Déplacer le backend (opération à faire une seule fois)

Depuis la racine du dépôt, et après avoir vérifié que tout est commité :

```
mkdir backend
git mv pom.xml backend/pom.xml
git mv src backend/src
....
git commit -m "chore: deplacer le backend dans backend/"
```

2.3 Le fichier `.gitignore` à la racine

Un seul `.gitignore` à la racine suffit pour les deux applications.

```
# Java / Maven
backend/target/
```

```
*.class

# Node / React
frontend/node_modules/
frontend/dist/
frontend/.env
frontend/.env.local

# Editeurs et systeme
.idea/
.vscode/
.DS_Store
Thumbs.db
```

Ne jamais commiter `node_modules/`, `target/` ni un fichier `.env` contenant des secrets.

2.4 Stratégie de branches

- `main` : version stable, toujours démarrable.
- `feature/frontend-stock` : une fonctionnalité frontend.
- `feature/backend-cors` : une modification backend.

Une branche porte sur *un seul* domaine. Cela évite qu'un commit frontend casse le backend et inversement.

3 Créer le projet React

3.1 Création avec Vite

Vite est l'outil recommandé aujourd'hui pour créer un projet React. Il est beaucoup plus rapide que l'ancien `create-react-app`, qui n'est plus maintenu.

Depuis la racine du dépôt :

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

Ouvrez ensuite http://localhost:5173. Vous devez voir la page d'accueil React. Pour arrêter le serveur : `Ctrl + C`.

3.2 Structure générée par Vite

```
frontend/
|-- index.html
|-- package.json
|-- vite.config.js
|-- public/
`-- src/
    |-- App.jsx
    |-- main.jsx
    |-- App.css
    |-- index.css
    `-- assets/
```

Vous allez réorganiser `src/` dans la section suivante. Ne modifiez pas `index.html` sauf pour le nom de l'application.

3.3 Commandes npm utiles

Commande	Rôle
`npm install`	installe les dépendances du `package.json`
`npm run dev`	lance le serveur de développement (port 5173)

3.4 Configurer le proxy de développement (indispensable)

En développement, React tourne sur le port `5173` et le backend sur le port `8080`. Ce sont deux *origines* différentes : le navigateur bloque donc les appels directs pour des raisons de sécurité (CORS).

La solution la plus simple consiste à demander à Vite de relayer les appels `/api` vers le backend. Modifiez `frontend/vite.config.js` :

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
```

```
      // Tout appel vers /api/... est transmis a http://localhost:8080/...
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''),
      },
    },
  },
})
```

Avec ce réglage, le frontend appelle par exemple `/api/produits`. Vite le transforme en `http://localhost:8080/p` Le navigateur ne voit qu'une seule origine (`localhost:5173`) : il n'y a plus de blocage CORS en développement.

3.5 CORS : le point critique

En développement, le proxy Vite de la section 3.5 règle le problème. Mais si vous appelez directement `http://localhost:8080/produits` (sans `/api`), le navigateur refusera la réponse :

```
Access to fetch at 'http://localhost:8080/produits' from origin
'http://localhost:5173' has been blocked by CORS policy.
```

La raison est simple : le `Router.java` actuel dans vos projets n'envoie aucun en-tête CORS et ne répond pas aux requêtes `OPTIONS` (le « preflight »). Pour autoriser un appel direct, ajoutez ceci dans `Router.java`.

1. Au tout début de la méthode `dispatcher`, ajoutez les en-têtes et le traitement de `OPTIONS` :

```
public void dispatcher(HttpExchange exchange) throws IOException {
    ajouterEntetesCors(exchange);

    // Preflight : le navigateur demande la permission avant POST/PUT/DELETE.
    if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
        exchange.sendResponseHeaders(204, -1);
        exchange.close();
        return;
    }

    String methode = exchange.getRequestMethod();
    String chemin = exchange.getRequestURI().getPath();
    String[] segments = chemin.split("/");
    try {
        Reponse reponse = router(methode, segments, exchange);
        envoyer(exchange, reponse.statut(), reponse.corps());
    } catch (ResourceNotFoundException e) {
        envoyer(exchange, 404, messageErreur(e));
    } catch (IllegalArgumentException e) {
        envoyer(exchange, 400, messageErreur(e));
    } catch (Exception e) {
        envoyer(exchange, 500, messageErreur(e));
    }
}

private void ajouterEntetesCors(HttpExchange exchange) {
    exchange.getResponseHeaders().add("Access-Control-Allow-Origin", "*");
    exchange.getResponseHeaders().add(
            "Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
    exchange.getResponseHeaders().add(
            "Access-Control-Allow-Headers", "Content-Type, Authorization");
```

```
}
```

4 Architecture du frontend

4.1 Arborescence recommandée

```
frontend/src/
|-- api/                  <-- communication HTTP (une seule porte vers le backend)
|   |-- client.js         <-- fetch + gestion des erreurs + URL de base
|   |-- produits.js
|   |-- fournisseurs.js
|   |-- categories.js
|   `-- utilisateurs.js
|-- components/           <-- composants reutilisables
|   |-- layout/
|   |   |-- Navbar.jsx
|   |   `-- Layout.jsx
|   |-- produits/
|   |   |-- ProduitCard.jsx
|   |   |-- ProduitList.jsx
|   |   `-- ProduitForm.jsx
|   `-- ui/
|       |-- Bouton.jsx
|       `-- Chargement.jsx
|-- pages/                <-- une page = une route de l'application
|   |-- ProduitsPage.jsx
|   |-- FournisseursPage.jsx
|   |-- FournisseurDetailPage.jsx
|   |-- CategoriesPage.jsx
|   `-- UtilisateursPage.jsx
|-- hooks/                <-- logique reutilisable (chargement, formulaires)
|   |-- useProduits.js
|   `-- useFournisseurs.js
|-- context/              <-- etat global eventuel (utilisateur connecte, theme)
|   `-- AuthContext.jsx
|-- utils/                <-- fonctions pures (dates, tri, constantes)
|   |-- dates.js
|   `-- constantes.js
|-- assets/               <-- images, icones
|-- styles/               <-- CSS global ou modules
|-- App.jsx               <-- routes
|-- main.jsx              <-- point d'entree React
`-- index.css
```

4.2 Rôle de chaque dossier

Dossier	Rôle et règle
`api/`	Tous les appels réseau. Un fichier par ressource (`produits.js`, `fournisseurs.js`...). Aucun composant ne doit appeler `fetch` directement.
`components/`	Composants réutilisables, sans logique métier lourde. Un composant = un fichier, nom en `PascalCase`.
`pages/`	Assemblage des composants pour une URL donnée. Les pages appellent les hooks, pas `fetch`.

`hooks/`	Logique réutilisable : chargement, pagination, formulaire. Nom toujours préfixé par `use`.
`context/`	État partagé par toute l'application. À utiliser seulement si plusieurs pages ont besoin de la même donnée.
`utils/`	Fonctions pures, testables, sans React (formatage de dates, constantes des seuils de stock).
`assets/`	Images et polices importées par le bundler.
`styles/`	CSS global, variables de thème, modules CSS.

Règle d'or : le sens des dépendances va toujours dans le même sens :

`pages/` → `hooks/` → `api/` → `fetch` → backend

Un composant de `components/` ne doit jamais importer une page, ni contenir l'URL du backend.

4.3 Ajouter le routage

Installez React Router :

```
npm install react-router-dom
```

Puis déclarez les routes dans `App.jsx` :

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import Layout from './components/layout/Layout'
import ProduitsPage from './pages/ProduitsPage'
import FournisseursPage from './pages/FournisseursPage'
import FournisseurDetailPage from './pages/FournisseurDetailPage'

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<Layout />}>
          <Route index element={<ProduitsPage />} />
          <Route path="fournisseurs" element={<FournisseursPage />} />
          <Route path="fournisseurs/:id" element={<FournisseurDetailPage />} />
        </Route>
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

`main.jsx` reste minimal :

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

4.4 Conventions de nommage

Élément	Convention	Exemple
Composant / page	PascalCase	`ProduitList.jsx`
Hook	camelCase préfixé `use`	`useProduits.js`
Service API	camelCase + `Api`	`produitsApi`
Fonction utilitaire	camelCase	`formaterDate`
Constante	SCREAMING_SNAKE_CASE	`SEUILS_STOCK`
Variable / état	camelCase	`chargement`

5 Communiquer avec le backend

5.1 Principe : le frontend ne connaît que HTTP et JSON

React ne parle pas directement à MySQL et n'importe aucune classe Java. Le frontend envoie une requête HTTP, reçoit du JSON, et affiche le résultat :

$$\text{React} \xrightarrow{\text{HTTP + JSON}} \text{Router} \rightarrow \text{Controller} \rightarrow \text{Service} \rightarrow \text{DAO} \rightarrow \text{MySQL}$$

Pour cet exemple de gestion de stock, le backend expose les ressources suivantes en JSON. Les noms de routes doivent être identiques dans le Router Java et dans React.

5.2 Contrat de l'API de gestion de stock

Le frontend utilise les ressources principales suivantes : produits, fournisseurs, utilisateurs et catégories.

Méthode	URL	Corps attendu	Réponse
GET	`/produits`	—	200, liste de produits
GET	`/produits/{id}`	—	200, un produit ; 404
POST	`/produits`	`ProduitRequest`	201, produit créé
PUT	`/produits/{id}`	`ProduitRequest`	200, produit modifié
DELETE	`/produits/{id}`	—	204, sans corps
GET	`/fournisseurs`	—	200, liste de fournisseurs
GET	`/fournisseurs/{id}`	—	200, un fournisseur ; 404
POST	`/fournisseurs`	`Fournisseur`	201, fournisseur créé
PUT	`/fournisseurs/{id}`	`Fournisseur`	200, fournisseur modifié
DELETE	`/fournisseurs/{id}`	—	204
GET	`/utilisateurs`	—	200, liste d'utilisateurs
GET	`/utilisateurs/{id}`	—	200, un utilisateur ; 404
POST	`/utilisateurs`	`Utilisateur`	201, utilisateur créé
PUT	`/utilisateurs/{id}`	`Utilisateur`	200
DELETE	`/utilisateurs/{id}`	—	204
GET	`/categories`	—	200, liste de catégories
GET	`/categories/{id}`	—	200, une catégorie ; 404
POST	`/categories`	`Categorie`	201, catégorie créée
PUT	`/categories/{id}`	`Categorie`	200
DELETE	`/categories/{id}`	—	204

Codes d'erreur : le backend renvoie `400` pour une donnée invalide, `404` pour une ressource absente et `500` pour une erreur serveur. Le corps est toujours de la forme :

```
{
  "erreur": "Produit introuvable : 99"
}
```

5.3 Tester avec Postman avant de coder React

Avant de toucher au frontend, validez chaque route avec Postman :

1. Méthode `GET`, URL `http://localhost:8080/produits`.
2. Pour un `POST` ou un `PUT`, onglet *Body* → *raw* → *JSON*, et en-tête `Content-Type: application/json`.
3. Vérifiez le code de statut HTTP et le corps JSON.

Si Postman échoue, React échouera aussi. Corrigez d'abord le backend.

5.4 La couche API du frontend

Créez `frontend/src/api/client.js`. C'est le *seul* fichier qui connaît l'URL du backend et la manière d'envoyer une requête.

```
const API_URL = import.meta.env.VITE_API_URL ?? '/api'

async function requete(path, options = {}) {
  const { headers, ...reste } = options

  const reponse = await fetch(`${API_URL}${path}`, {
    ...reste,
    headers: {
      'Content-Type': 'application/json',
      ...headers,
    },
  })

  // Un DELETE reussi renvoie 204 : pas de corps a lire.
  if (reponse.status === 204) {
    return null
  }

  const texte = await reponse.text()
  let donnees = null
  if (texte) {
    try {
      donnees = JSON.parse(texte)
    } catch {
      donnees = null
    }
  }

  if (!reponse.ok) {
    const message = donnees?.erreur ?? texte ?? `Erreur HTTP ${reponse.status}`
    throw new Error(message)
  }

  return donnees
}

export const api = {
  get: (path) => requete(path),
```

```
  post: (path, body) => requete(path, { method: 'POST', body: JSON.stringify(body) }),
  put: (path, body) => requete(path, { method: 'PUT', body: JSON.stringify(body) }),
  delete: (path) => requete(path, { method: 'DELETE' }),
}
```

Puis un service par ressource. Exemple pour les produits (`frontend/src/api/produits.js`) :

```
import { api } from './client'

export const produitsApi = {
  lister: () => api.get('/produits'),
  obtenir: (id) => api.get(`/produits/${id}`),
  creer: (produit) => api.post('/produits', produit),
  modifier: (id, produit) => api.put(`/produits/${id}`, produit),
  supprimer: (id) => api.delete(`/produits/${id}`),
}
```

Même principe pour `fournisseurs.js`, `categories.js` et `utilisateurs.js` :

```
import { api } from './client'

export const fournisseursApi = {
  lister: () => api.get('/fournisseurs'),
  obtenir: (id) => api.get(`/fournisseurs/${id}`),
  creer: (fournisseur) => api.post('/fournisseurs', fournisseur),
  modifier: (id, fournisseur) => api.put(`/fournisseurs/${id}`, fournisseur),
  supprimer: (id) => api.delete(`/fournisseurs/${id}`),
}

export const categoriesApi = {
  lister: () => api.get('/categories'),
  creer: (categorie) => api.post('/categories', categorie),
  modifier: (id, categorie) => api.put(`/categories/${id}`, categorie),
  supprimer: (id) => api.delete(`/categories/${id}`),
}
```

5.5 Un hook pour charger les données

Le hook centralise les trois états indispensables :
chargement, *erreur*, *données*. Créez `frontend/src/hooks/useProduits.js` :

```
import { useCallback, useEffect, useState } from 'react'
import { produitsApi } from '../api/produits'

export function useProduits(categorieId) {
  const [produits, setProduits] = useState([])
  const [chargement, setChargement] = useState(true)
  const [erreur, setErreur] = useState(null)

  const recharger = useCallback(async () => {
    try {
      setChargement(true)
      setErreur(null)
      const donnees = await produitsApi.lister()
      const resultat = categorieId
        ? donnees.filter((produit) => produit.categorieId === Number(categorieId))
        : donnees
      setProduits(resultat)
    } catch (e) {
```

```
      setErreur(e.message)
    } finally {
      setChargement(false)
    }
  }, [categorieId])

  useEffect(() => {
    recharger()
  }, [recharger])

  return { produits, chargement, erreur, recharger }
}
```

5.6 Une page complète : liste + création + suppression

frontend/src/components/produits/ProduitList.jsx :

```
import { produitsApi } from '../../api/produits'

function ProduitList({ produits, onSuppression }) {
  const supprimer = async (id) => {
    if (!window.confirm('Supprimer ce produit ?')) return
    try {
      await produitsApi.supprimer(id)
      onSuppression()
    } catch (e) {
      alert(e.message)
    }
  }

  if (produits.length === 0) {
    return <p>Aucun produit pour le moment.</p>
  }

  return (
    <ul>
      {produits.map((produit) => (
        <li key={produit.id}>
          <strong>{produit.nom}</strong> - Ref. {produit.reference}
          : {produit.quantite} unites ({produit.prixUnitaire} DT)
          <button onClick={() => supprimer(produit.id)}>Supprimer</button>
        </li>
      ))}
    </ul>
  )
}

export default ProduitList
```

frontend/src/components/produits/ProduitForm.jsx :

```
import { useState } from 'react'
import { produitsApi } from '../../api/produits'

const VALEURS_INITIALES = {
  nom: '',
  reference: '',
  description: '',
  quantite: '0',
```

```
  prixUnitaire: '',
  seuilAlerte: '5',
  categorieId: '',
  fournisseurId: '1',
}

function ProduitForm({ onCreation }) {
  const [form, setForm] = useState(VALEURS_INITIALES)
  const [erreur, setErreur] = useState(null)

  const changer = (e) => {
    const { name, value } = e.target
    setForm((ancien) => ({ ...ancien, [name]: value }))
  }

  const soumettre = async (e) => {
    e.preventDefault()
    setErreur(null)
    try {
      await produitsApi.creer({
        ...form,
        quantite: Number(form.quantite),
        prixUnitaire: Number(form.prixUnitaire),
        seuilAlerte: Number(form.seuilAlerte),
        fournisseurId: Number(form.fournisseurId),
        categorieId: form.categorieId ? Number(form.categorieId) : null,
      })
      setForm(VALEURS_INITIALES)
      onCreation()
    } catch (err) {
      setErreur(err.message)
    }
  }

  return (
    <form onSubmit={soumettre}>
      <input name="nom" placeholder="Nom du produit" value={form.nom}
        onChange={changer} required />
      <input name="reference" placeholder="Reference" value={form.reference}
        onChange={changer} required />
      <textarea name="description" placeholder="Description"
        value={form.description} onChange={changer} />
      <input type="number" name="quantite" min="0" placeholder="Quantite"
        value={form.quantite} onChange={changer} required />
      <input type="number" name="prixUnitaire" min="0" step="0.01"
        placeholder="Prix unitaire" value={form.prixUnitaire}
        onChange={changer} required />
      <input type="number" name="seuilAlerte" min="0"
        placeholder="Seuil d'alerte" value={form.seuilAlerte}
        onChange={changer} required />
      <input type="number" name="categorieId" min="1"
        placeholder="Identifiant categorie" value={form.categorieId}
        onChange={changer} />
      <input type="number" name="fournisseurId" min="1"
        value={form.fournisseurId} onChange={changer} required />
      {erreur && <p className="erreur">{erreur}</p>}
      <button type="submit">Ajouter</button>
    </form>
  )
}
```

```

export default ProduitForm
```

`frontend/src/pages/ProduitsPage.jsx` assemble le tout :

```jsx
import { useProduits } from '../hooks/useProduits'
import ProduitList from '../components/produits/ProduitList'
import ProduitForm from '../components/produits/ProduitForm'

function ProduitsPage() {
  const { produits, chargement, erreur, recharger } = useProduits()

  if (chargement) return <p>Chargement...</p>
  if (erreur) return <p className="erreur">Erreur : {erreur}</p>

  return (
    <section>
      <h1>Gestion du stock</h1>
      <ProduitForm onCreation={recharger} />
      <ProduitList produits={produits} onSuppression={recharger} />
    </section>
  )
}

export default ProduitsPage
```

5.7 Gérer les dates

Le backend n'utilise pas un format unique :

Source	Format	Exemple
`dateAjout` (produit)	`yyyy-MM-dd`	`2026-07-15`
`dateInscription` (utilisateur)	`yyyy-MM-dd HH:mm:ss`	`2026-06-01 09:00:00`
`dateCreation`, `createdAt`, `updatedAt` (fournisseur)	ISO	`2026-06-01T09:00:00`
`dateFin` (fournisseur)	`yyyy-MM-dd`	`2026-12-31`

Écrivez un utilitaire robuste dans `frontend/src/utils/dates.js` :

```jsx
export function formaterDate(valeur, options) {
  if (!valeur) return '-'

  // "2026-06-01 09:00:00" n'est pas un ISO valide pour tous les navigateurs.
  const normalise = valeur.includes('T') ? valeur : valeur.replace(' ', 'T')
  const date = new Date(normalise)

  if (Number.isNaN(date.getTime())) return valeur

  return date.toLocaleDateString('fr-FR', options)
}
```

Pour un champ `<input type="date">`, envoyez toujours `yyyy-MM-dd`. Ne convertissez pas avec `toISOString()` sans réfléchir au décalage horaire : cela peut décaler la date d'un jour.

Le frontend ne remplace pas le backend : il le consomme.
Une route validée dans Postman est une route prête à être branchée dans React.